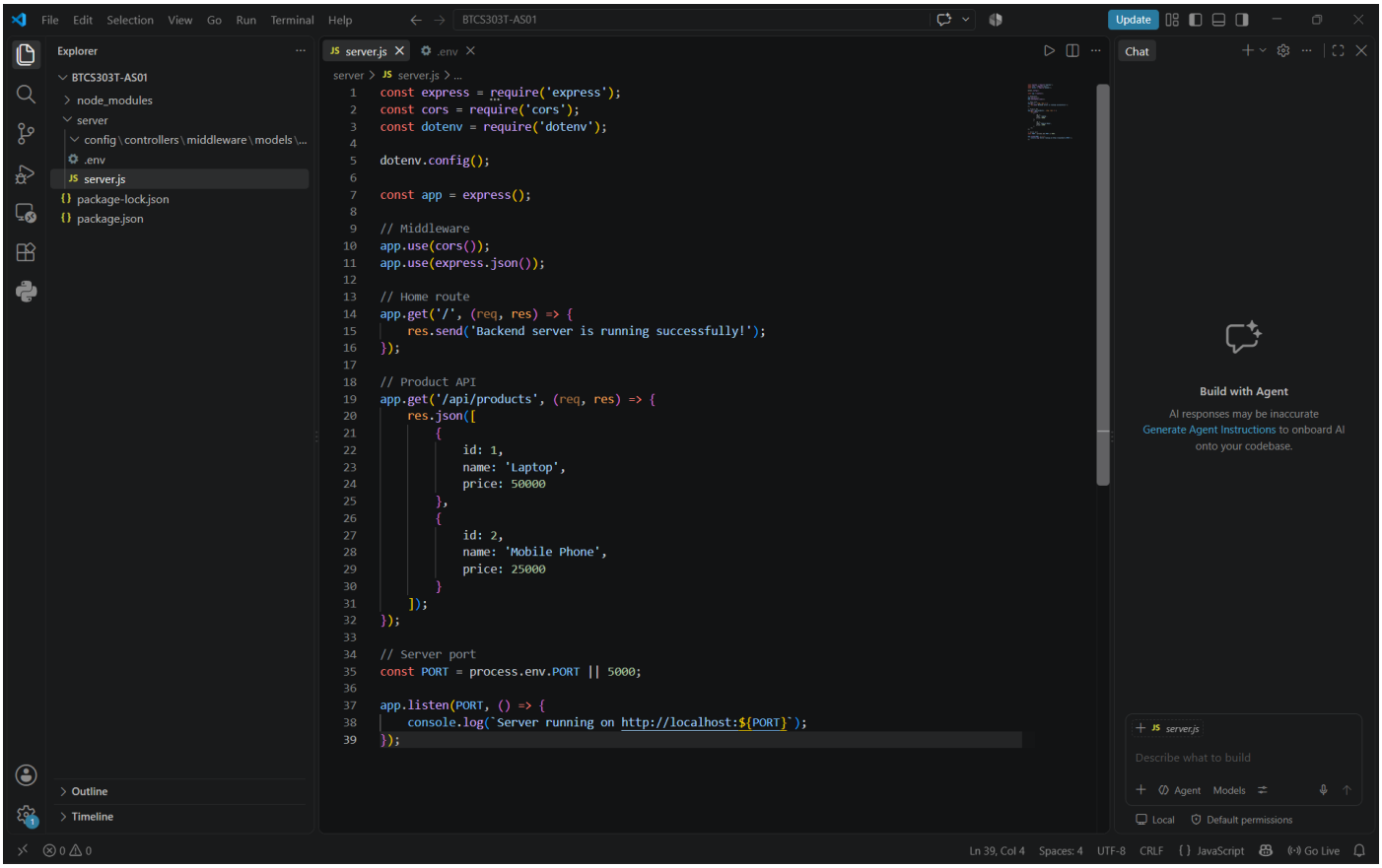
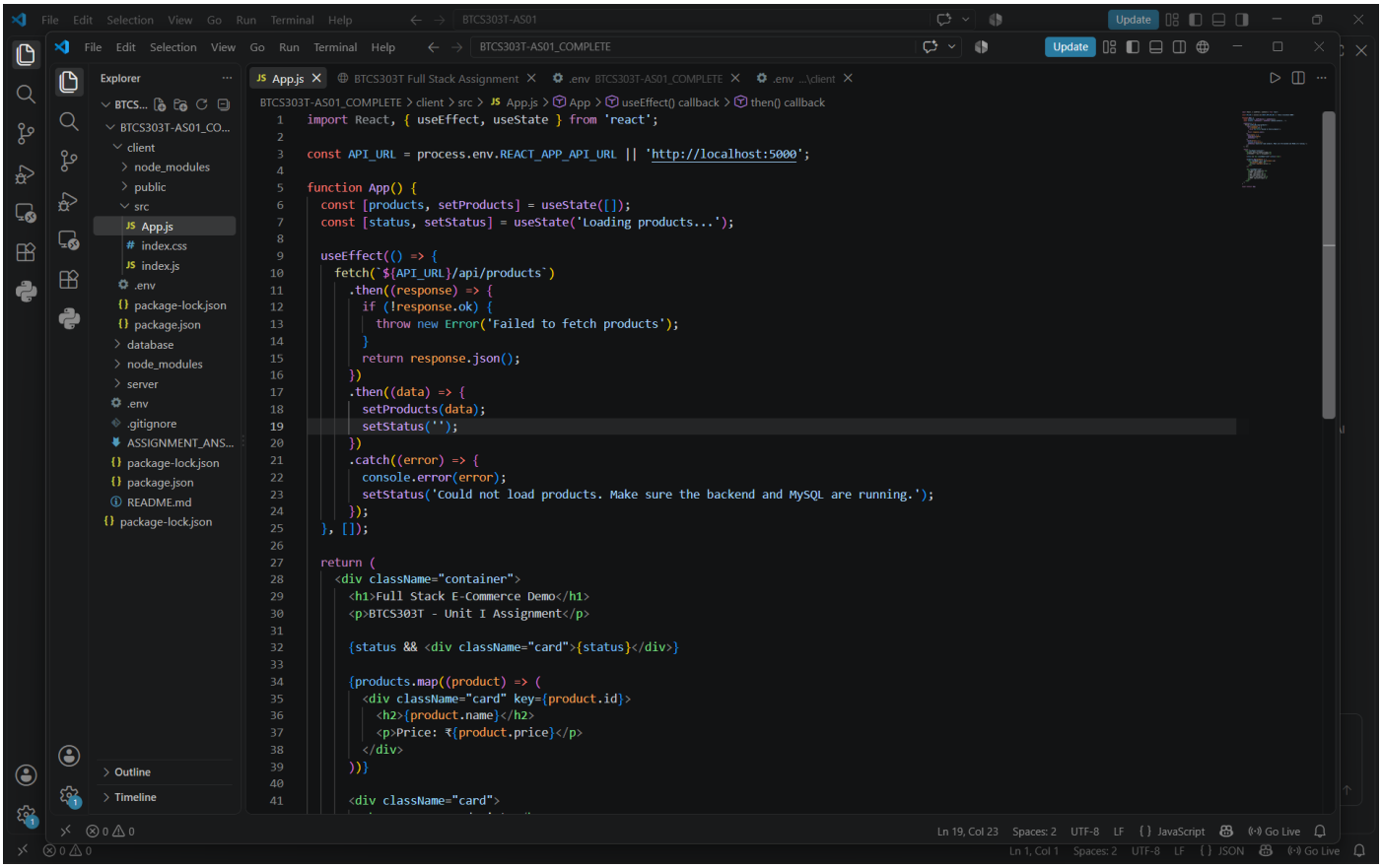
Plain text

```
BTCS303T-AS01_COMPLETE
|
├── client
|   ├── src
|   |   └── App.js
|   └── .env
|
├── database
|   └── setup.sql
|
├── server
|   ├── config
|   |   └── db.js
|   ├── routes
|   |   ├── authRoutes.js
|   |   └── bookRoutes.js
|   └── server.js
|
├── .env
├── .gitignore
└── package.json
```





```
import React, { useEffect, useState } from 'react';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000';

function App() {
  const [products, setProducts] = useState([]);
  const [status, setStatus] = useState('Loading products...');

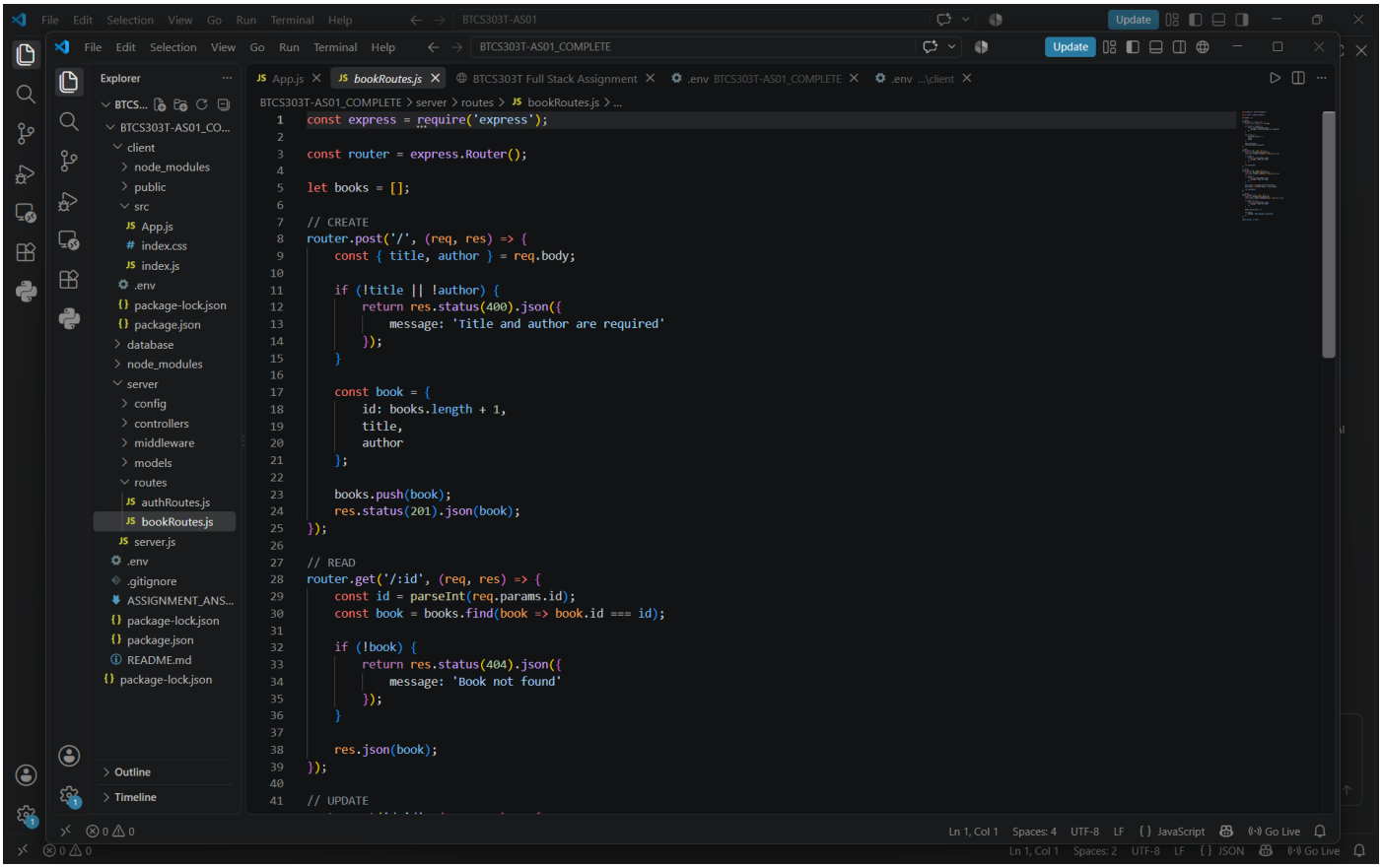
  useEffect(() => {
    fetch(`${API_URL}/api/products`)
      .then((response) => {
        if (!response.ok) {
          throw new Error('Failed to fetch products');
        }
        return response.json();
      })
      .then((data) => {
        setProducts(data);
        setStatus('');
      })
      .catch((error) => {
        console.error(error);
        setStatus('Could not load products. Make sure the backend and MySQL are running.');
      });
  }, []);

  return (
    <div className="container">
      <h1>Full Stack E-Commerce Demo</h1>
      <p>BTCS303T - Unit I Assignment</p>

      {status && <div className="card">{status}</div>}

      {products.map((product) => (
        <div className="card" key={product.id}>
          <h2>{product.name}</h2>
          <p>Price: ₹{product.price}</p>
        </div>
      ))}

      <div className="card">
```



```
const express = require('express');

const router = express.Router();

let books = [];

// CREATE
router.post('/', (req, res) => {
    const { title, author } = req.body;

    if (!title || !author) {
        return res.status(400).json({
            message: 'Title and author are required'
        });
    }

    const book = {
        id: books.length + 1,
        title,
        author
    };

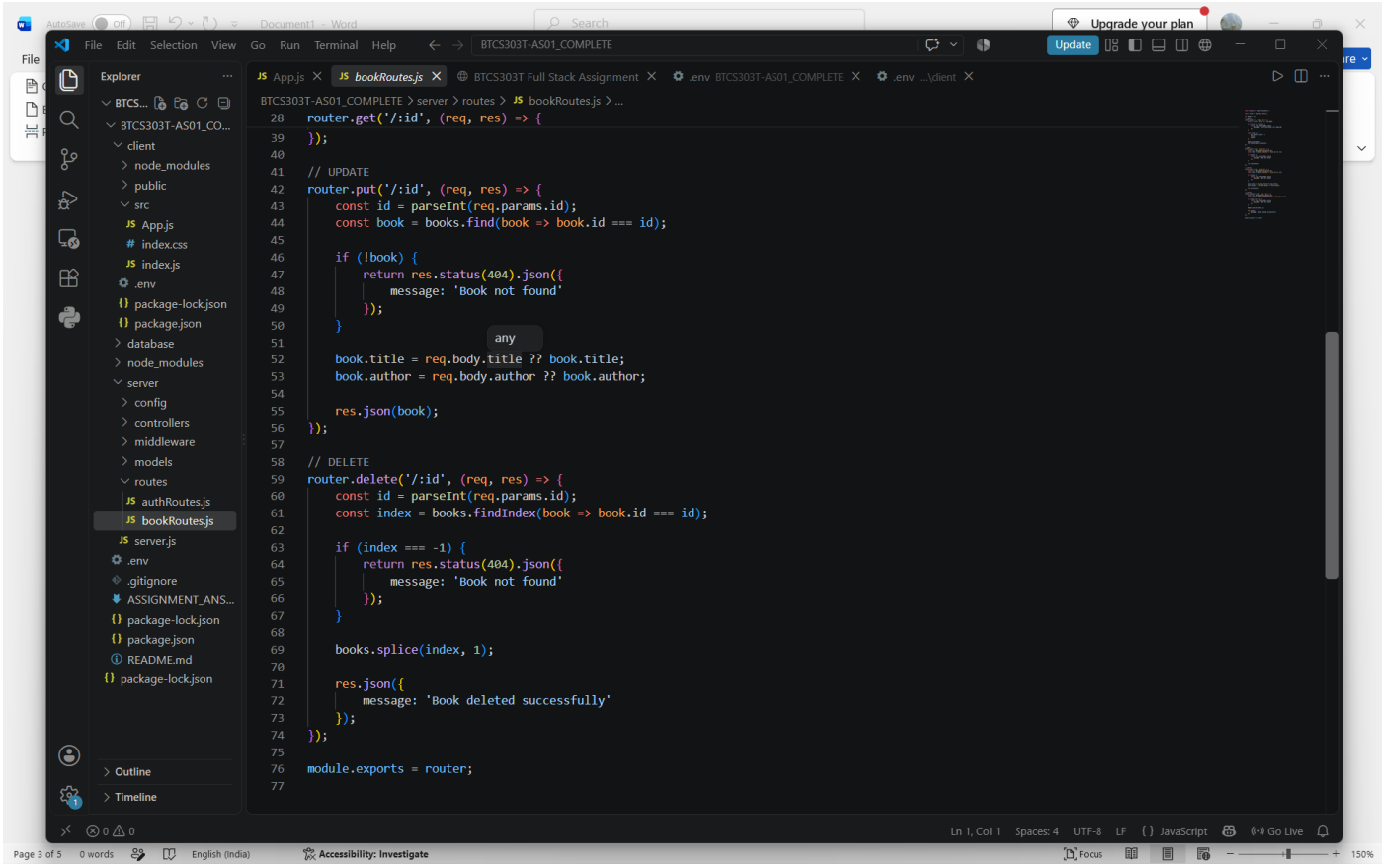
    books.push(book);
    res.status(201).json(book);
});

// READ
router.get('/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const book = books.find(book => book.id === id);

    if (!book) {
        return res.status(404).json({
            message: 'Book not found'
        });
    }

    res.json(book);
});

// UPDATE
```



```
router.get('/:id', (req, res) => {
});

// UPDATE
router.put('/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const book = books.find(book => book.id === id);

    if (!book) {
        return res.status(404).json({
            message: 'Book not found'
        });
    }
                        any
    book.title = req.body.title ?? book.title;
    book.author = req.body.author ?? book.author;

    res.json(book);
});

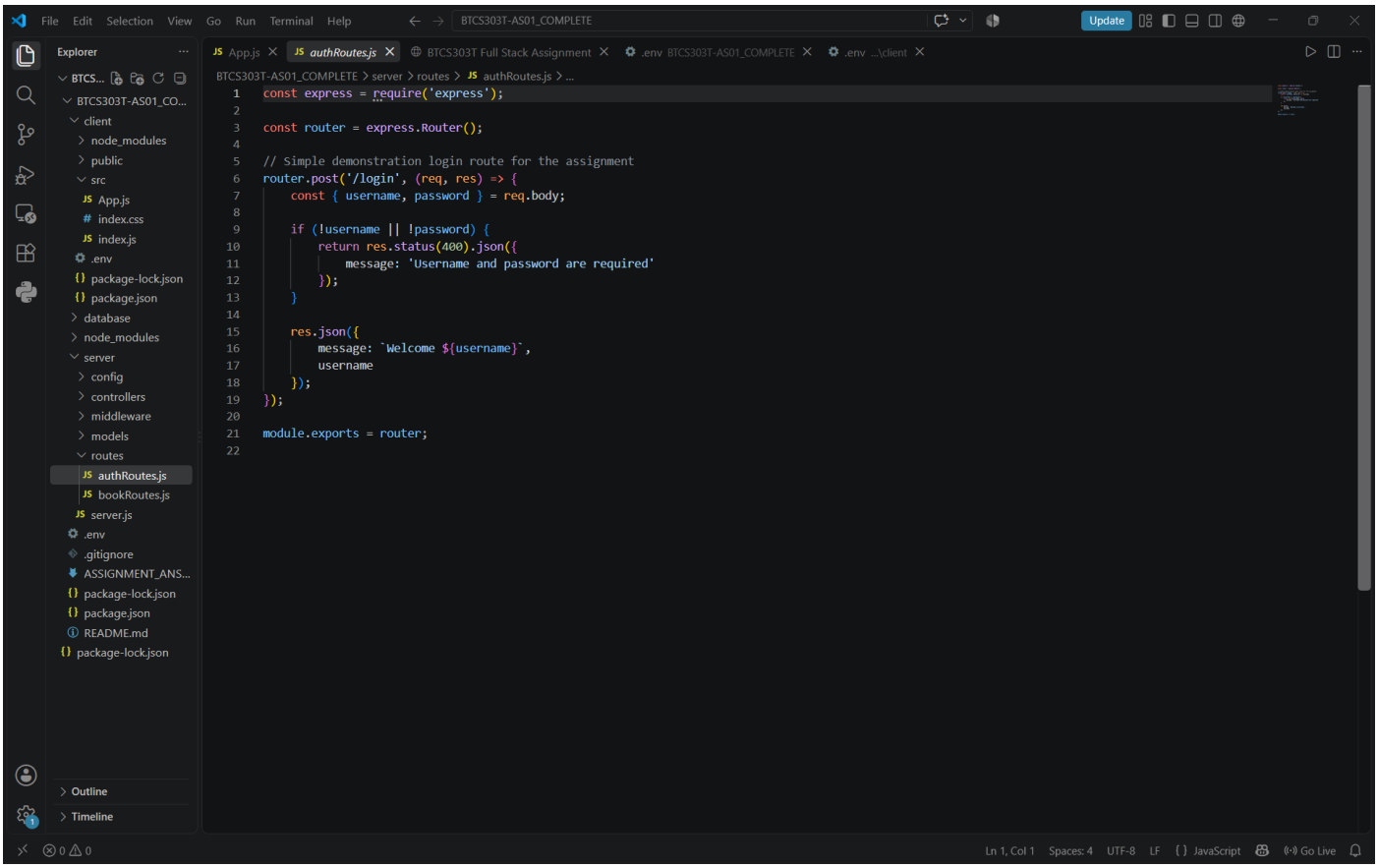
// DELETE
router.delete('/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = books.findIndex(book => book.id === id);

    if (index === -1) {
        return res.status(404).json({
            message: 'Book not found'
        });
    }

    books.splice(index, 1);

    res.json({
        message: 'Book deleted successfully'
    });
});

module.exports = router;
```



```
const express = require('express');

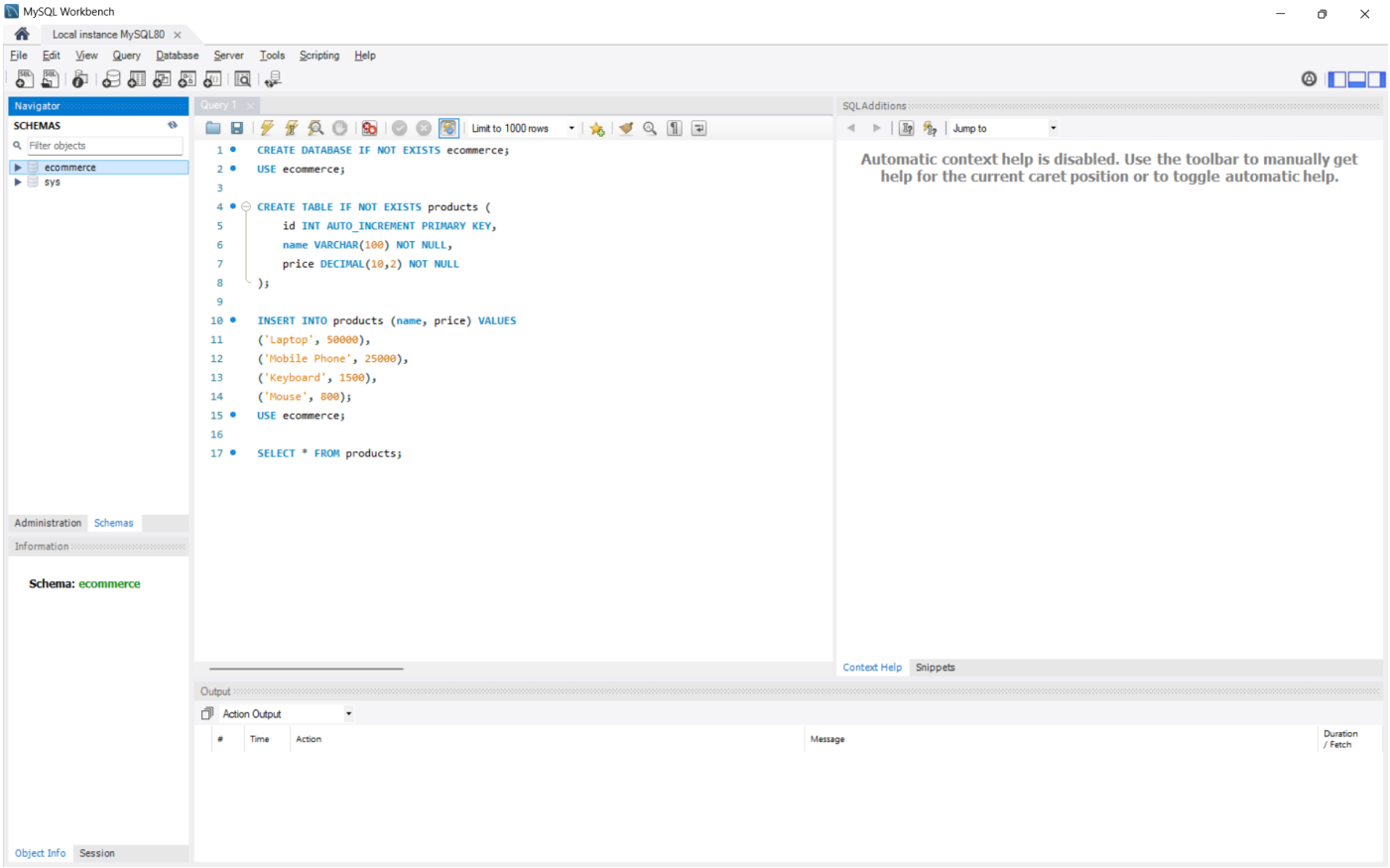
const router = express.Router();

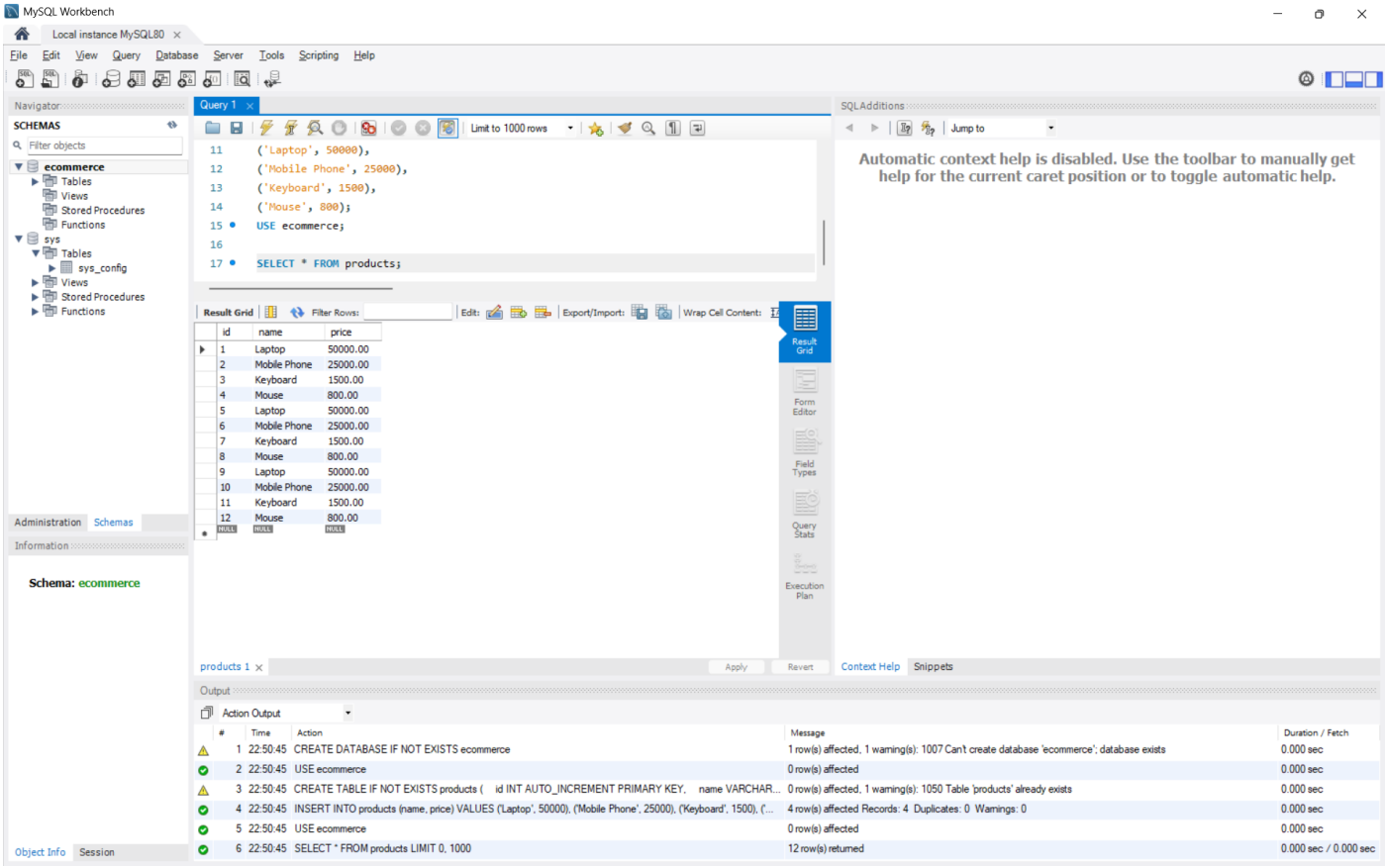
// Simple demonstration login route for the assignment
router.post('/login', (req, res) => {
    const { username, password } = req.body;

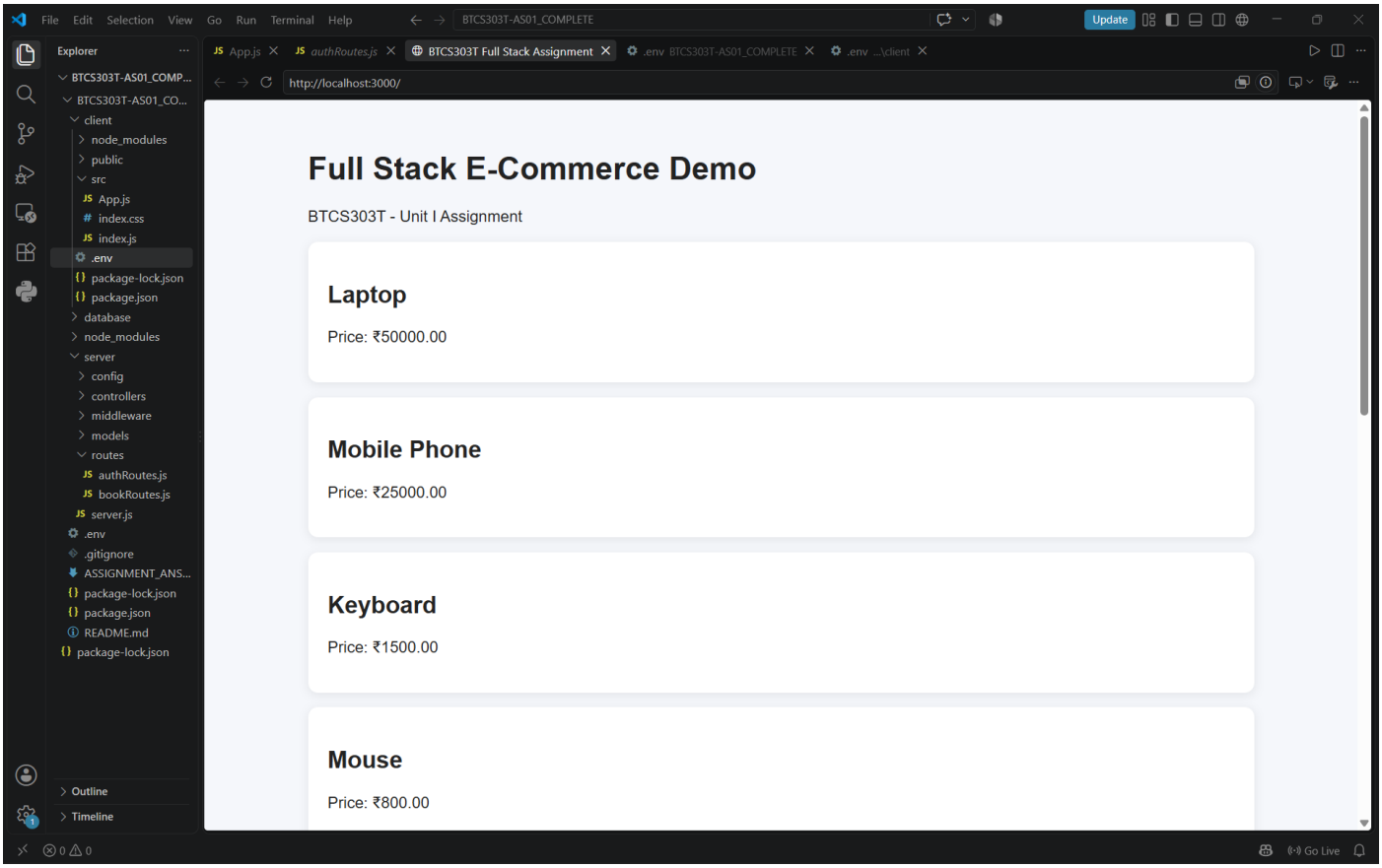
    if (!username || !password) {
        return res.status(400).json({
            message: 'Username and password are required'
        });
    }

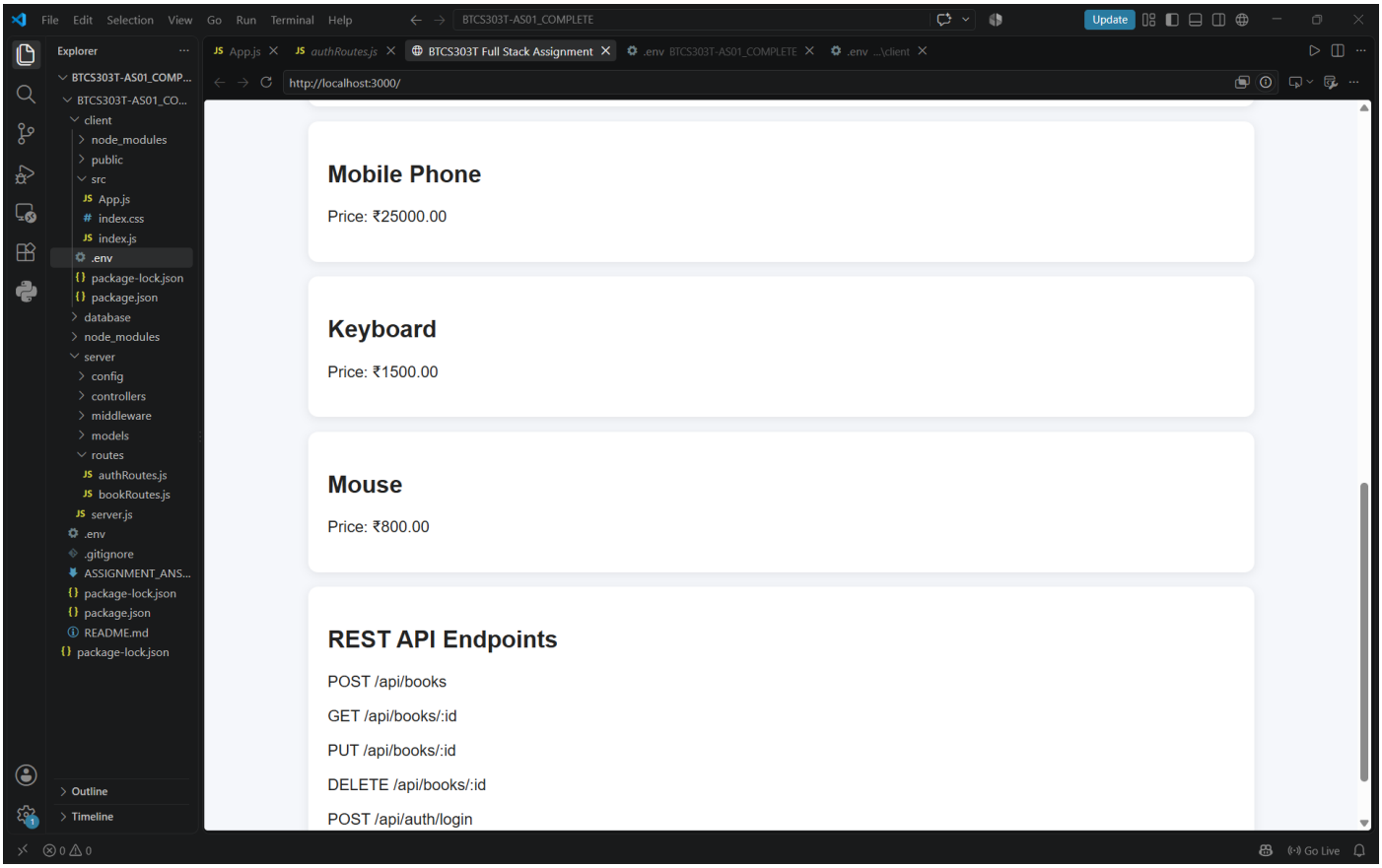
    res.json({
        message: `Welcome ${username}`,
        username
    });
});

module.exports = router;
```


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
ecommerce
sys
Query 1
Limit to 1000 rows
1 CREATE DATABASE IF NOT EXISTS ecommerce;
2 USE ecommerce;
3
4 CREATE TABLE IF NOT EXISTS products (
5 id INT AUTO_INCREMENT PRIMARY KEY,
6 name VARCHAR(100) NOT NULL,
7 price DECIMAL(10,2) NOT NULL
8);
9
10 INSERT INTO products (name, price) VALUES
11 ('Laptop', 50000),
12 ('Mobile Phone', 25000),
13 ('Keyboard', 1500),
14 ('Mouse', 800);
15 USE ecommerce;
16
17 SELECT * FROM products;
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Administration Schemas
Information
Schema: ecommerce
Object Info Session
Output
Action Output
Time Action Message Duration / Fetch


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
ecommerce
Tables
Views
Stored Procedures
Functions
sys
Tables
sys_config
Views
Stored Procedures
Functions
Query 1
Limit to 1000 rows
('Laptop', 50000),
('Mobile Phone', 25000),
('Keyboard', 1500),
('Mouse', 800);
USE ecommerce;
SELECT * FROM products;
Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:
id name price
1 Laptop 50000.00
2 Mobile Phone 25000.00
3 Keyboard 1500.00
4 Mouse 800.00
5 Laptop 50000.00
6 Mobile Phone 25000.00
7 Keyboard 1500.00
8 Mouse 800.00
9 Laptop 50000.00
10 Mobile Phone 25000.00
11 Keyboard 1500.00
12 Mouse 800.00
Result Grid
Form Editor
Field Types
Query Stats
Execution Plan
products 1
Apply Revert
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Administration Schemas
Information
Schema: ecommerce
Object Info Session
Output
Action Output
Time Action Message Duration / Fetch
1 22:50:45 CREATE DATABASE IF NOT EXISTS ecommerce 1 row(s) affected, 1 warning(s): 1007 Can't create database 'ecommerce'; database exists 0.000 sec
2 22:50:45 USE ecommerce 0 row(s) affected 0.000 sec
3 22:50:45 CREATE TABLE IF NOT EXISTS products (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR... 0 row(s) affected, 1 warning(s): 1050 Table 'products' already exists 0.000 sec
4 22:50:45 INSERT INTO products (name, price) VALUES ('Laptop', 50000), ('Mobile Phone', 25000), ('Keyboard', 1500), ('... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0 0.000 sec
5 22:50:45 USE ecommerce 0 row(s) affected 0.000 sec
6 22:50:45 SELECT * FROM products LIMIT 0, 1000 12 row(s) returned 0.000 sec / 0.000 sec


File Edit Selection View Go Run Terminal Help
BTCS303T-AS01_COMPLETE
Update
Explorer
App.js
authRoutes.js
BTCS303T Full Stack Assignment
.env BTCS303T-AS01_COMPLETE
.env ...\client
http://localhost:3000/
BTCS303T-AS01_COMP...
client
node_modules
public
src
App.js
index.css
index.js
.env
package-lock.json
package.json
database
node_modules
server
config
controllers
middleware
models
routes
authRoutes.js
bookRoutes.js
server.js
.env
.gitignore
ASSIGNMENT_ANS...
package-lock.json
package.json
README.md
package-lock.json
Outline
Timeline
Go Live
Full Stack E-Commerce Demo
BTCS303T - Unit I Assignment
Laptop
Price: ₹50000.00
Mobile Phone
Price: ₹25000.00
Keyboard
Price: ₹1500.00
Mouse
Price: ₹800.00


File Edit Selection View Go Run Terminal Help
BTCS303T-AS01_COMPLETE
Update
Explorer
App.js
authRoutes.js
BTCS303T Full Stack Assignment
.env BTCS303T-AS01_COMPLETE
.env ...\client
http://localhost:3000/
Mobile Phone
Price: ₹25000.00
Keyboard
Price: ₹1500.00
Mouse
Price: ₹800.00
REST API Endpoints
POST /api/books
GET /api/books/:id
PUT /api/books/:id
DELETE /api/books/:id
POST /api/auth/login